



Project Documentation and Deep Review

Codebase-derived documentation for the EduCore Learning Management System and Digital Management System, covering charter, business requirements, SRS, stories, use cases, architecture, database, API, UI/UX, security, testing, DevOps, deployment, maintenance, project management, changelog, and user guidance.

PROJECT

EduCore LMS & DMS

DOCUMENT STATUS

Code Review Draft

REVIEWED ON

May 26, 2026

REPOSITORY

C:\xampp\htdocs\EduCore

Basis of review: README, existing docs, sprint plan, backend Express modules, MySQL schema, Mongoose models, OpenAPI summary, frontend routes and pages, Docker Compose, environment template, and TypeScript validation. Items not present in the repository are marked as assumptions or recommended additions.

00 Table Of Contents

1. Project Charter / Vision Document	2. Business Requirements Document
3. Software Requirements Specification	4. User Stories
5. Use Case Documentation	6. System Architecture Document
7. Database Documentation	8. API Documentation
9. UI/UX Documentation	10. Security Documentation
11. Testing Documentation	12. DevOps Documentation
13. Deployment Guide	14. Maintenance Documentation
15. Project Management Documentation	16. Change Log
17. User Documentation	18. Technical Documentation Structure

01 Project Charter / Vision Document

Item	Project Definition
Project Name	EduCore LMS & DMS
Project Description	EduCore is a full-stack platform that combines learning management and digital management operations for schools, bootcamps, training providers, or university departments. It provides role-based portals, course/class management, admissions, documents, lessons, assignments, attendance, gradebook, certificates, quizzes, notifications, search, reporting, import/export, and an AI-style Copilot workspace.
Business Goals	Centralize educational operations, reduce manual administration, improve instructor and learner visibility, support digital admissions and document review, and provide reporting for academic decision making.
Problem Statement	Educational teams often manage applications, classes, attendance, assignments, lesson content, documents, communications, and reports in disconnected tools. This causes duplicate work, weak visibility, slow student support, and inconsistent records. EduCore addresses this by combining operational records and learning workflows in one portal.
Target Users	Administrators, instructors, students, public applicants, academic leadership, admissions staff, IT/deployment maintainers, and project evaluators.
Stakeholders	Project sponsor, product owner, academic administration, instructors, learners, admissions/review team, support staff, developers, database administrator, DevOps maintainer, security reviewer.

Success Metrics (KPIs)

KPI	Target	Measurement Method
Role coverage	Admin, instructor, student, and public applicant flows complete for MVP.	Acceptance test checklist and demo scripts.
Administrative efficiency	Reduce manual student/course/attendance/report work by at least 40 percent.	Compare manual workflow steps before and after adoption.
Page response	Initial portal pages load in under 2 seconds on local/staging network.	Lighthouse or browser performance traces.

EduCore LMS & DMS Enterprise Documentations

Review draft - 2026-05-26

API performance	Common list/detail endpoints under 300 ms p95 for demo scale.	API load tests and server logs.
Data quality	Unique emails, student codes, and enrollment constraints prevent duplicate key records.	Database constraints and negative tests.
Security readiness	Zero known exposed secrets and zero critical dependency vulnerabilities before production.	Secret scanning, dependency audit, and security review.
Testing	Automated tests cover auth, RBAC, core APIs, and major UI workflows.	CI test reports.
Operational readiness	Backup, restore, logging, monitoring, and deployment runbooks documented.	Maintenance checklist sign-off.

Scope

- JWT authentication and role-based authorization.
- Admin operations for users, students, instructors, applications, courses, classes, enrollments, documents, CMS, reports, certificates, and import/export.
- Instructor operations for class content, assignments, grading, attendance, quizzes, success center, notifications, and class analytics.
- Student operations for dashboard, lessons, assignments, submissions, documents, gradebook/certificates, quizzes, notifications, recommendations, and Copilot.
- Public course catalog and course application submission.
- MySQL plus MongoDB persistence, Socket.IO realtime events, and Docker Compose local deployment.

Out Of Scope

- Payment processing, invoicing, payroll, and finance modules.
- Native iOS/Android apps.
- External SIS/LTI integrations, enterprise SSO, and SCORM/xAPI runtime.
- Production queue workers, Redis cache, CDN, object storage migration, and full-text search engine.
- Advanced proctoring, plagiarism detection, and video conferencing.
- Multi-tenant institution isolation unless added as a future phase.

Budget

The repository does not define a formal monetary budget. Recommended planning assumption: treat the current version as an academic or MVP project with internal labor and low-cost local infrastructure. A production pilot should budget for hosting, managed MySQL/MongoDB or VM capacity, object storage, domain/TLS, email delivery, monitoring, backups, and security review. Expected MVP infrastructure range is approximately USD 50-300/month depending on managed services and traffic.

Timeline

The repository includes a six-sprint plan: setup, authentication, DMS, LMS, advanced features, and finalization. A realistic capstone/MVP timeline is 6-10 weeks with one sprint per week or per two weeks. Production hardening should be planned as a separate phase after MVP acceptance.

02 Business Requirements Document (BRD)

Executive Summary

EduCore is designed to replace fragmented school operations with one authenticated portal. It serves administrators who need operational control, instructors who need class delivery tools, students who need learning and communication workflows, and applicants who need a public application path. The project already implements a broad MVP surface, but production readiness requires stronger automated tests, secret hygiene, CI/CD, monitoring, and backup procedures.

Business Objectives

Objective	Business Rationale	Implemented Evidence
Increase operational visibility	Admins need dashboards, reports, and searchable records.	Reports routes, dashboard page, search page, success center.
Reduce manual work	Manual spreadsheets and disconnected records cause mistakes.	Student/instructor CRUD, enrollment, attendance bulk save, import/export.
Improve student experience	Students need one place for classes, work, grades, documents, quizzes, and notifications.	Student home, lessons, work, gradebook, documents, quiz and inbox flows.
Improve instructor workflow	Instructors need content, attendance, grading, quiz, and intervention tools.	Instructor home, Class Studio, assignments, attendance, live quizzes, success center.
Support admissions pipeline	Applicants need public course discovery; admins need review and enrollment handoff.	Public course/application routes and admin Applications page.
Enable analytics and support	At-risk students need early intervention.	Success center and class/attendance/performance reports.

Functional Goals

- User registration, login, logout, refresh token rotation, and current user lookup.
 - Role-based portal navigation for admin, instructor, and student.
 - User, student, instructor, course, class, enrollment, assignment, submission, attendance, document,
- notification, CMS, quiz, application, report, search, and import/export workflows.
 - Public course browsing and public course application submission.
 - Realtime notifications and live quiz updates using Socket.IO.
 - AI-style Copilot chat, drafts, documents, citations, and audit access.

Non-Functional Goals

Category	Goal	Current Support	Gap
----------	------	-----------------	-----

Performance	Fast portal and API response for MVP scale.	Pagination helpers, SQL indexes, TanStack Query caching.	No load tests yet.
Security	Protect data by role, validate input, hash passwords, rate limit requests.	JWT, bcrypt, RBAC, Zod, Helmet, CORS, rate limit.	Secret leakage risk, localStorage tokens, missing upload MIME allowlist.
Scalability	Keep modules separated and deployable.	Backend route modules, separate frontend/backend/Docker services.	No queue/cache/search service yet.
Reliability	Recover from database or app failures.	Docker service restart policies.	No backup/restore automation in repo.
Maintainability	Use typed code and module boundaries.	TypeScript in frontend/backend; typecheck passes.	No automated unit/integration test suite.
Usability	Support efficient role-specific workflows.	Role-aware navigation and responsive UI components.	No formal usability testing artifacts.

03 Software Requirements Specification (SRS)

Introduction

Area	Specification
Purpose	Define the functional and non-functional requirements for the EduCore LMS & DMS system.
Scope	A role-based web platform for educational operations, learning delivery, admissions, reporting, documents, notifications, quizzes, and AI-assisted support.
Definitions	LMS: Learning Management System. DMS: Digital Management System. RBAC: Role-Based Access Control. JWT: JSON Web Token. CMS: Content Management System. KPI: Key Performance Indicator.
References	README.md , docs/project-plan.md , docs/erd.md , docs/api-docs.md , backend/docs/openapi.yaml , backend routes, database schema, frontend routes.

Overall Description

Product Perspective

EduCore is a monorepo with a React/Vite frontend and an Express/TypeScript backend. MySQL stores relational academic records. MongoDB stores flexible content, notifications, documents, quiz sessions, applications, and Copilot artifacts.

User Classes

Public Applicant

Student

Instructor

Admin

Maintainer

Constraints

Node.js, Express, React, TypeScript, MySQL, MongoDB, local uploads, JWT auth, Docker Compose service model, and browser-based UI.

Assumptions

Production deployment will replace development secrets, configure SMTP/object storage, enable HTTPS, add CI/CD and backup jobs, and define actual budget and service-level targets.

Functional Requirements

ID	Feature	Description	Inputs	Outputs	Acceptance Criteria
----	---------	-------------	--------	---------	---------------------

FR-001	Public Course Catalog	Visitors can view published courses and available class metadata.	Optional filters.	Course cards.	Only published/visible courses are listed; course title, instructor, level, and class count appear.
FR-002	Course Application	Visitors submit admissions applications.	Name, email, phone, course, education, message.	Application record.	Required fields validate; duplicate/review state is trackable by admin.
FR-003	User Registration	Users can register as students through the auth API.	Full name, email, password, profile fields.	User, student profile, tokens.	Email is unique; password is at least 8 characters; linked student profile is created.
FR-004	Login	Users authenticate using email and password.	Email, password.	Access token, refresh token, public user.	Inactive or invalid credentials return 401; active users receive tokens.
FR-005	Session Refresh/Logout	Refresh tokens rotate and can be revoked.	Refresh token.	New token pair or revoked session.	Expired/revoked tokens fail; logout revokes matching token hash.
FR-006	RBAC	Protected routes enforce admin, instructor, and student authorization.	JWT access token.	Allowed response or 401/403.	Unauthorized users cannot access restricted modules.
FR-007	User Management	Admins manage base user accounts and status.	Name, email, role, status, password.	User records.	Admins can list, create, update, and deactivate/delete users.
FR-008	Student Management	Admins manage student profiles and class assignments; instructors can view permitted students.	Student code, department, semester, classes.	Student list/detail/history.	Student code and email remain unique; class assignments persist.
FR-009	Instructor Management	Admins manage instructor profiles and class assignments.	Profile fields, specialization, classes.	Instructor records.	Instructor accounts can be created, updated, password reset, assigned, and removed.
FR-010	Course Management	Admins create and maintain courses.	Title, description, instructor, level, status.	Course catalog entries.	Draft/published/archived states are supported.

FR-011	Class Scheduling	Admins and instructors manage class room, schedule, start/end dates, and class days.	Course, room, schedule JSON, days.	Class and class-day records.	Classes link to courses; days are unique per class/day number.
FR-012	Enrollment	Admins enroll students into classes.	Student ID, class ID.	Enrollment row.	A student cannot be enrolled twice in the same class.
FR-013	Class Studio	Instructors/admins manage class-day content, comments, backups, and assignments.	Class day content, blocks, assets, comments.	Published days and backups.	Assigned instructors can edit their class content; backups preserve snapshots.
FR-014	Lesson CMS	Admins create dynamic lessons and assets.	Course, title, content, blocks, assets, published flag.	Lesson documents.	Students see published content for enrolled classes.
FR-015	Assignments	Admins/instructors create assignments attached to courses/classes/days.	Title, description, due date, points.	Assignment records and notifications.	Students can view assigned work; invalid due dates fail validation.
FR-016	Submissions	Students upload assignment submissions.	Assignment ID, file, notes.	Submission record.	Duplicate assignment/student submissions are blocked; file size is limited to 10 MB.
FR-017	Grading	Instructors grade submissions and provide feedback.	Submission ID, grade, feedback.	Updated submission.	Grades are numeric and visible in gradebook/performance reports.
FR-018	Attendance	Instructors mark class-day attendance, including bulk save.	Class, day, students, status, notes.	Attendance records.	Only instructors can mark attendance; status is one of present, absent, late, excused.
FR-019	Gradebook	Users view gradebook rows according to role.	Class filter.	Performance rows.	Students see their data; admins/instructors see permitted classes.
FR-020	Certificates	Admins/instructors issue certificates; admins configure templates and revoke.	Student, class, template elements.	Certificate record and verification code.	Certificate eligibility is calculated; revoked certificates remain auditable.
FR-021	Documents	Students/admins upload documents; admins approve/reject.	Title, type, file.	Student document records.	Students see their own documents; admins can review all.

FR-022	Notifications	Admins/instructors send realtime and persisted notifications.	Title, message, type, user/role target.	Notification document and socket event.	Recipients receive messages in inbox without full page refresh.
FR-023	Reports	Admins/instructors view dashboard, attendance, performance, success center, and class analytics.	Filters and role context.	Aggregated report data.	Restricted data respects role; dashboards show totals and trends.
FR-024	Global Search	Admins/instructors search across SQL and MongoDB resources.	Search query.	Grouped results.	Empty or short queries are handled; results grouped by entity.
FR-025	Import/Export	Admins import students and export key entities.	CSV/XLSX file or export format.	Created records or CSV/JSON/XLSX.	Invalid rows are rejected or reported; exports include requested entity.
FR-026	Live Quizzes	Instructors generate/start/close quizzes; students accept and submit attempts.	Class/day, questions, answers.	Quiz sessions and attempts.	Scores calculate correctly; student cannot submit duplicate attempt.
FR-027	Recommendations	Students receive lesson recommendations from weak performance signals.	Student performance.	Recommended lessons.	Non-students receive an empty or restricted response.
FR-028	Copilot	Users chat with contextual assistant; drafts/documents/audit are managed by role.	Message, mode, documents, draft status.	Threads, messages, drafts, citations.	Role visibility rules apply; admin can view audit data.
FR-029	CMS Pages/Announcements	Admins manage portal pages; admins/instructors post announcements.	Slug, title, blocks, status, audience.	CMS content and announcements.	Slug is unique; audience visibility is enforced.
FR-030	Student Success	Admins/instructors track at-risk students and interventions.	Student, note, priority, action.	Intervention/activity records.	Risk list reflects attendance/grade/submission indicators.

Non-Functional Requirements

ID	Requirement	Acceptance Criteria
NFR-001	Performance	Core API list endpoints use pagination where relevant and return under agreed p95 target for MVP data volume.

NFR-002	Security	All protected endpoints require JWT; role checks are tested; production secrets are not committed.
NFR-003	Validation	All write endpoints validate body, query, and params using schemas or equivalent checks.
NFR-004	Reliability	Database connections fail fast on startup; service health endpoint is available at <code>/health</code> .
NFR-005	Maintainability	Backend modules remain route/service/domain separated; TypeScript check passes in CI.
NFR-006	Usability	Portal navigation is role-aware and responsive for desktop and mobile.
NFR-007	Observability	Production deployment includes structured logs, error tracking, metrics, and alerting.
NFR-008	Backup	MySQL, MongoDB, and uploaded files have scheduled backups and documented restore tests.

04 User Stories

ID	Story	Acceptance Criteria	Priority	Points
US-001	As a visitor, I want to browse courses so that I can choose a program before applying.	Published courses appear; course level and instructor are visible; application action is available.	High	5
US-002	As an applicant, I want to submit a course application so that admissions can review me.	Required data validates; application is stored; admin can see it in Admissions.	High	8
US-003	As an admin, I want to manage student records so that learner information stays accurate.	Create/update/deactivate works; email and code are unique; class assignments persist.	High	8
US-004	As an admin, I want to manage instructors so that teaching assignments are tracked.	Instructor account/profile creation works; classes can be assigned; password can be reset.	High	8
US-005	As an admin, I want to create courses and classes so that the institution can publish offerings.	Course stores instructor/status; class stores schedule; class appears in portal.	High	8
US-006	As an instructor, I want to edit class day content so that students receive structured lesson material.	Class day can be created/updated; published state controls visibility; assets can be attached.	High	8
US-007	As an instructor, I want to create assignments so that students can submit work.	Assignment requires title/due date/points; students see work; notification is generated.	High	5
US-008	As a student, I want to upload assignment files so that my instructor can grade them.	Upload validates size; submission is linked to assignment and student; duplicate is blocked.	High	5
US-009	As an instructor, I want to grade submissions so that student performance is visible.	Grade and feedback save; gradebook updates; student can view result.	High	5
US-010	As an instructor, I want to mark attendance in bulk so that class attendance is efficient.	Roster loads; statuses save; summary reflects present/absent/late/excused counts.	High	8

US-011	As an admin, I want dashboard and reports so that I can monitor school operations.	Dashboard totals, attendance, performance, class analytics, and success center load by role.	High	8
US-012	As a student, I want a personalized home page so that I can see courses, work, notifications, and lessons quickly.	Home page shows enrolled classes, assignments, notifications, and published class days.	High	5
US-013	As an admin, I want to review student documents so that required documents are approved or rejected.	Documents upload; admin can approve/reject; student sees status.	Medium	5
US-014	As an instructor, I want to run live quizzes so that class understanding can be measured immediately.	Questions generate/start; students accept/submit; results and close action work.	Medium	8
US-015	As a student, I want recommendations so that I can revisit weak topics.	Recommendations are based on weak performance; irrelevant roles are restricted.	Medium	5
US-016	As an admin, I want import/export so that external records can move in and out of the system.	Students import from file; exports support CSV, JSON, and XLSX.	Medium	5
US-017	As any authenticated user, I want notifications so that I receive timely messages.	Notifications persist, mark read, and update through sockets.	Medium	5
US-018	As an instructor/admin, I want a success center so that at-risk students receive intervention.	Risk data loads; intervention notes save; activity is auditable.	Medium	8

05 Use Case Documentation

Use Case	Actors	Preconditions	Main Flow	Alternate Flow	Postconditions
UC-001 Apply For Course	Visitor	Public course exists.	Open public home; select course; enter applicant data; submit application.	Validation fails or course unavailable.	Application exists with pending/new status.
UC-002 Login	Admin, instructor, student	User account is active.	Open login; enter email/password; system validates; tokens are stored; role portal opens.	Invalid credentials return error; inactive user rejected.	Authenticated session exists.
UC-003 Manage Student	Admin	Admin is logged in.	Open Students; create or edit profile; assign classes; save.	Duplicate email/code rejected.	Student profile and related user are stored.
UC-004 Create Course And Class	Admin	Instructor exists.	Open Courses; create course; create class with room/schedule; publish as needed.	Missing instructor or invalid schedule.	Course and class are available to permitted users.
UC-005 Enroll Student	Admin	Student and class exist.	Select class; select student; submit enrollment.	Duplicate enrollment blocked.	Enrollment row exists.
UC-006 Publish Class Day	Instructor, admin	Class exists and user is authorized.	Open Class Studio; select class; create or edit class day; attach assets; publish.	Unauthorized instructor cannot edit unrelated class.	ClassDay document exists and can appear to students.
UC-007 Submit Assignment	Student	Student has visible assignment.	Open Work; choose file; upload submission.	File exceeds 10 MB or duplicate submission exists.	Submission record exists and instructor queue updates.
UC-008 Grade Submission	Instructor	Submission exists.	Open Assignments/Class Studio; select submission; enter grade and feedback; save.	Invalid grade rejected.	Gradebook and reports reflect grade.
UC-009 Mark Attendance	Instructor	Class roster exists.	Open Attendance; choose class day; mark statuses; save bulk attendance.	Outside edit window or invalid status.	Attendance records exist for class day.
UC-010 Run Live Quiz	Instructor, student	Class day has quiz questions.	Instructor starts session; socket notifies students; students accept and submit; instructor reviews results.	Student misses timer or session closes.	Quiz session and attempts are stored.

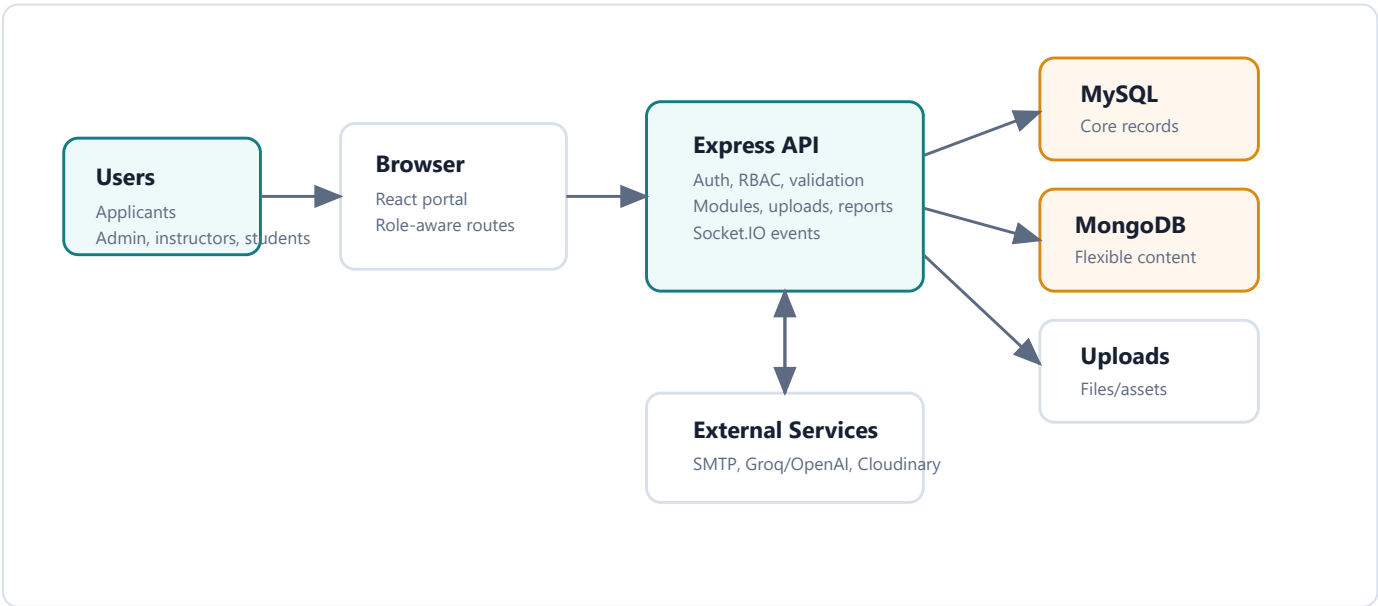
UC-011 Issue Certificate	Admin, instructor	Student is eligible.	Open Gradebook; select eligible row; issue certificate.	Student not eligible or certificate already exists.	Certificate with verification code exists.
UC-012 Send Notification	Admin, instructor	User is authorized.	Open Notifications; enter title/message/target; send.	Invalid role/user target.	Notification persists and socket event is emitted.

06 System Architecture Document

Architecture Overview

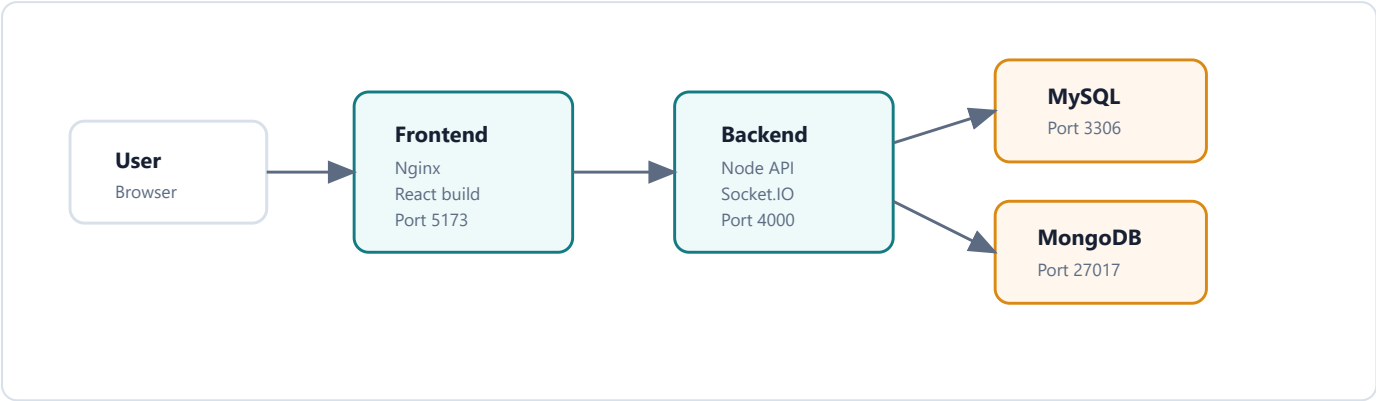
Layer	Technology	Responsibility
Frontend	React 19, Vite 6, TypeScript, Tailwind CSS, React Router, TanStack Query, React Hook Form, Zod, Recharts, Lucide, Socket.IO Client	Role-based portal UI, public home, forms, dashboards, tables, reports, realtime inbox/quiz updates.
Backend	Node.js, Express 4, TypeScript, JWT, bcryptjs, Zod, Multer, Socket.IO, Swagger UI, Nodemailer	REST API, authentication, RBAC, validation, uploads, email, realtime events, reports, import/export.
Relational Database	MySQL 8 / XAMPP MySQL	Users, students, instructors, courses, classes, enrollments, attendance, assignments, submissions, refresh tokens.
Document Database	MongoDB 7 via Mongoose	Lessons, class days, notifications, applications, documents, certificates, Copilot data, CMS content, quiz sessions/attempts.
Storage	Local uploads folder, optional Cloudinary env placeholders	Student documents, submissions, CMS/Copilot assets.
External Services	SMTP, optional Groq/OpenAI-compatible API, optional Cloudinary	Email notifications/admissions; Copilot model provider; future object storage.

Architecture Diagram



Deployment Architecture

The repository includes Docker Compose services for MySQL, MongoDB, backend, and frontend. The frontend container serves the built React app through Nginx on port 5173. The backend container exposes port 4000 and depends on database services. Local development also supports XAMPP MySQL and a local MongoDB instance.

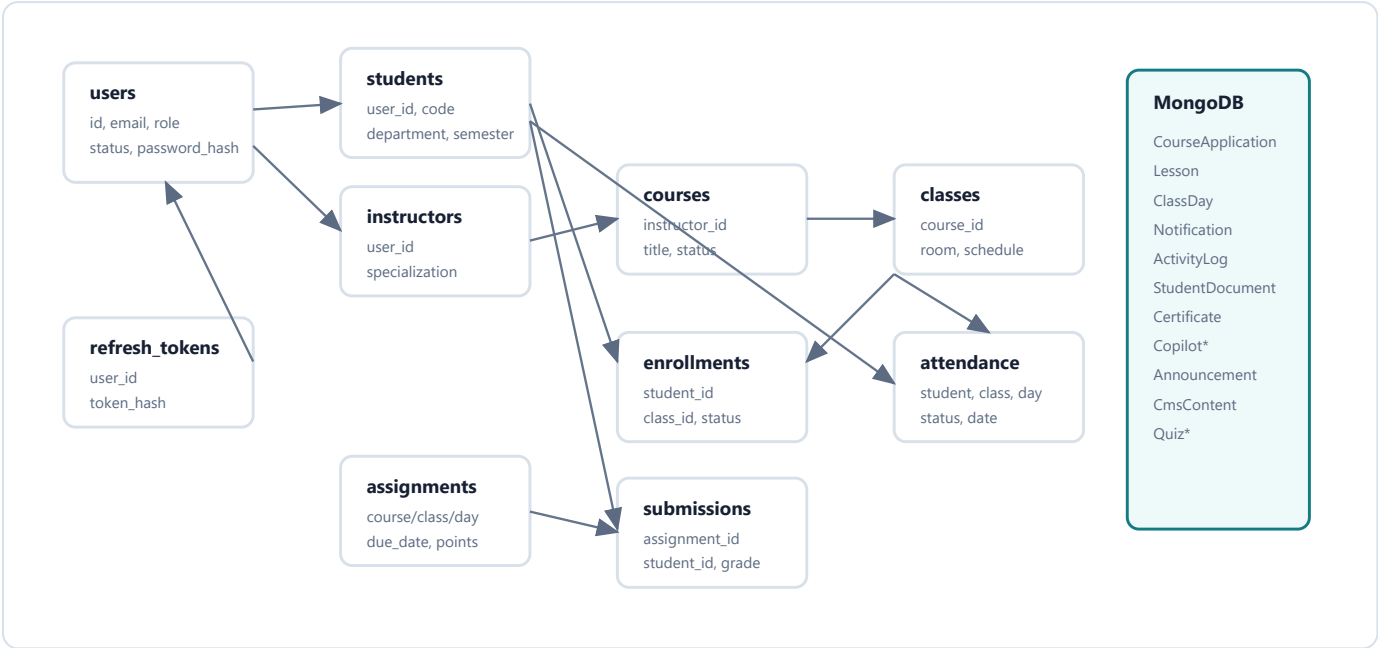


Architecture Decisions

Decision	Reason	Tradeoff
MySQL plus MongoDB	Relational records need constraints; content, applications, quiz and Copilot objects benefit from flexible schemas.	Two database engines increase backup and operational complexity.
JWT access plus refresh token table	Stateless access tokens with revocable refresh sessions.	Token storage must be hardened for production.
Socket.IO	Simple realtime events for notifications, submissions, and quiz updates.	Production scaling needs sticky sessions or adapter/backplane.
Local uploads	Simple MVP implementation for files.	Production should use object storage, antivirus scanning, and signed access.
Modular Express routes	Feature areas are discoverable and independently maintainable.	Service layer separation is partial; large route files should be decomposed later.

07 Database Documentation

ERD Summary



MySQL Table Documentation

Table	Primary Key	Important Fields	Relationships / Constraints
users	id CHAR(36)	full_name VARCHAR(120), email VARCHAR(160) UNIQUE, password_hash, role ENUM(admin,instructor,student), status ENUM(active,inactive), timestamps	Parent of students, instructors, refresh_tokens.
students	id CHAR(36)	user_id CHAR(36) UNIQUE, student_code VARCHAR(40) UNIQUE, department, semester, timestamps	FK users.id ON DELETE CASCADE. Parent of enrollments, attendance, submissions.
instructors	id CHAR(36)	user_id CHAR(36) UNIQUE, specialization, timestamps	FK users.id ON DELETE CASCADE. Parent of courses.
courses	id CHAR(36)	title, description, instructor_id, level, status ENUM(draft,published,archived), timestamps	FK instructors.id ON DELETE RESTRICT. Parent of classes and assignments.
classes	id CHAR(36)	course_id, room, schedule JSON, starts_at, ends_at, timestamps	FK courses.id ON DELETE CASCADE. Parent of enrollments and attendance.
enrollments	id CHAR(36)	student_id, class_id, status ENUM(active,completed,dropped), created_at	FK students.id and classes.id. Unique student_id + class_id.

attendance	id CHAR(36)	student_id, class_id, class_day_id, status ENUM(present,absent,late,excused), date, notes	FK students.id and classes.id. Unique student_id + class_id + class_day_id.
assignments	id CHAR(36)	course_id, class_id, class_day_id, title, description, due_date, points, timestamps	FK courses.id CASCADE and classes.id SET NULL.
submissions	id CHAR(36)	assignment_id, student_id, file_url, notes, grade DECIMAL(5,2), feedback, submitted_at, graded_at	FK assignments.id and students.id. Unique assignment_id + student_id.
refresh_tokens	id CHAR(36)	user_id, token_hash CHAR(64) UNIQUE, expires_at, revoked_at, created_at	FK users.id ON DELETE CASCADE. Stores hashed refresh tokens.

MongoDB Collection Documentation

Collection / Model	Purpose	Key Fields	Indexes / Notes
Lesson	Dynamic lessons.	courseId, title, content, blocks, assets, order, published.	courseId indexed.
ClassDay	Class-specific day content.	classId, courseId, dayNumber, title, content, blocks, assets, published, updatedBy.	Unique classId + dayNumber.
ClassBackup	Class content snapshots.	classId, title, days, createdBy.	classId indexed.
ClassComment	Instructor/admin class comments.	classId, message, authorId, authorName, authorRole, parentId.	Supports threaded comments.
Notification	Persisted notifications.	userId, role, title, message, type, metadata, readAt.	userId and role indexed.
ActivityLog	Audit/activity timeline.	userId, action, entity, entityId, metadata.	Used for seed/audit style records.
CourseApplication	Admissions applications.	fullName, email, courseId, courseTitle, status, stage, interviewAt, notes, enrolledClassId.	Email, courseId, status/stage indexed.
EmailLog	Email delivery records.	to, subject, category, status, providerMessageId, error, relatedEntity.	Useful for admissions timeline.
StudentDocument	Uploaded student documents.	studentId, userId, title, type, fileUrl, originalName, mimeType, size, status.	studentId, userId, type, status indexed.
Certificate	Issued certificates.	studentId, classId, courseId, finalGrade, verificationCode, status, issued/revoked metadata.	verificationCode unique.
CertificateTemplate	Certificate design data.	name, page, elements, updatedBy.	Stores editor snapshot.
CopilotThread	Copilot chat thread metadata.	userId, role, title, mode, lastMessageAt, archivedAt.	userId and role indexed.
EduCore LMS & DMS Enterprise Documentation			Review draft - 2026-05-26

CopilotMessage	Copilot chat messages.	threadId, userId, role, content, provider, citations, actions, metadata.	threadId + createdAt indexed.
CopilotDraft	AI-assisted draft workflow.	createdBy, approvedBy, type, title, content, status, targets.	Status/type indexed.
CopilotDocument	Uploaded Copilot reference documents.	uploadedBy, title, fileUrl, extractedText, visibility, targetRole, status.	Visibility/status indexed.
Announcement	Course/system announcements.	courseId, title, body, audience, publishedAt.	courseId indexed.
CmsContent	Editable pages.	slug, title, blocks, status, updatedBy.	slug unique.
QuizQuestion	Quiz question bank.	lessonId, classDayId, classId, courseId, prompt, type, options, correctAnswers.	Lesson/class indexes.
QuizSession	Live quiz session.	classId, courseId, dayId, status, questions, participants, startedBy, endedAt.	Status/class indexes.
QuizAttempt	Student quiz attempt.	sessionId, studentId, studentUserId, answers, score, total.	Unique sessionId + studentUserId.

08 API Documentation

Base API URL: `http://localhost:4000/api` . Swagger UI is mounted at `http://localhost:4000/api/docs` . All non-auth protected APIs expect `Authorization: Bearer <accessToken>` . Public endpoints exist under `/api/public` . Health check is `GET /health` .

Common Responses

Status	Meaning	Example Body
200/201	Successful operation or created resource.	<code>{ "data": [...] }</code> or entity JSON.
204	No content response for delete/no-body operations.	No body.
401	Missing, invalid, or expired access/refresh token.	<code>{ "message": "Invalid or expired access token" }</code>
403	User role is not allowed.	<code>{ "message": "You do not have permission..." }</code>
409	Duplicate key or conflict.	<code>{ "message": "Duplicate record" }</code>
422	Zod validation failure.	<code>{ "message": "Validation failed", "errors": {...} }</code>
500	Unhandled server error.	<code>{ "message": "Internal server error" }</code>

Endpoint Catalog

Group	Method / Endpoint	Roles	Description
Auth	POST /auth/register	Public	Register student user and profile.
Auth	POST /auth/login	Public	Authenticate with email/password.
Auth	POST /auth/refresh	Public	Rotate refresh token and issue access token.
Auth	POST /auth/logout	Public	Revoke refresh token.
Auth	GET /auth/me	Authenticated	Return current user.
Public	GET /public/courses	Public	List public courses.
Public	POST /public/course-applications	Public	Create course application.
Users	GET, POST /users	Admin	List/create users.
Users	PUT, DELETE /users/:id	Admin	Update/delete user.
Students	GET /students	Admin, instructor	List students.
Students	POST /students	Admin	Create student/user.
Students	GET /students/:id/history	Admin, instructor	Student timeline/history.
Students	PUT /students/:id	Admin	Update student and classes.

Students	POST /students/:id/reset-password	Admin	Reset student password.
Students	DELETE /students/:id	Admin	Delete student.
Instructors	GET /instructors	Admin, instructor	List instructors.
Instructors	POST /instructors	Admin	Create instructor/user.
Instructors	PUT /instructors/:id	Admin	Update instructor.
Instructors	POST /instructors/:id/reset-password	Admin	Reset instructor password.
Instructors	PUT /instructors/:id/class-assignment	Admin	Assign instructor to class.
Instructors	DELETE /instructors/:id	Admin	Delete instructor.
Applications	GET /course-applications	Admin	List admissions applications.
Applications	GET /course-applications/:id/timeline	Admin	Application email/audit timeline.
Applications	PATCH /course-applications/:id	Admin	Update application status/stage.
Applications	POST /course-applications/:id/enroll	Admin	Create student/enrollment from accepted application.
Courses	GET /courses	Authenticated	List visible courses.
Courses	POST, PUT, DELETE /courses	Admin	Create/update/delete courses.
Classes	GET /courses/classes/all	Authenticated	List classes visible by role.
Classes	GET, POST /courses/:id/classes	GET all auth, POST admin	List/create course classes.
Classes	PUT /courses/classes/:id	Admin, instructor	Update class.
Classes	GET /courses/classes/:id/students	Admin, instructor	Class roster.
Class Studio	GET /courses/classes/:id/days	Authenticated	Class days.
Class Studio	POST /courses/classes/:id/days	Admin, instructor	Create class day.
Class Studio	PUT /courses/classes/days/:dayId	Admin, instructor	Update class day.
Class Studio	GET, POST /courses/classes/:id/comments	Admin, instructor	Read/write class comments.
Class Studio	GET /courses/classes/:id/backups	Admin, instructor	List class backups.
Class Studio	POST /courses/classes/:id/backup	Admin, instructor	Create class backup.
Enrollment	POST /courses/classes/:id/enrollments	Admin	Enroll student.
Assignments	GET /assignments	Authenticated	List assignments by role.
Assignments	GET /assignments/class-health	Admin	Class assignment/grading health.

Assignments	POST /assignments/class-health/:id/notify-ungraded	Admin	Notify ungraded class.
Assignments	POST, PUT /assignments	Admin, instructor	Create/update assignments.
Assignments	DELETE /assignments/:id	Admin	Delete assignment.
Submissions	GET /assignments/submissions	Authenticated	List submissions by role.
Submissions	POST /assignments/submissions	Student, admin	Upload submission.
Submissions	PUT /assignments/submissions/:id/grade	Instructor	Grade submission.
Attendance	GET /attendance	Admin, instructor	List attendance records.
Attendance	GET /attendance/health	Admin, instructor	Attendance health by class.
Attendance	POST, PUT /attendance	Instructor	Create/update attendance.
Attendance	POST /attendance/bulk	Instructor	Bulk attendance save.
Attendance	GET /attendance/summary	Admin, instructor	Attendance summary.
Documents	GET /documents	Admin, student	List documents.
Documents	POST /documents	Admin, student	Upload document.
Documents	PATCH /documents/:id/review	Admin	Approve/reject document.
Gradebook	GET /gradebook	Admin, instructor, student	Gradebook rows.
Gradebook	GET /gradebook/certificates	Admin, instructor, student	List certificates by role.
Gradebook	GET, PUT /gradebook/certificate-template	GET all roles, PUT admin	Read/update certificate template.
Gradebook	POST /gradebook/certificates	Admin, instructor	Issue certificate.
Gradebook	PATCH /gradebook/certificates/:id/revoke	Admin	Revoke certificate.
Notifications	GET /notifications	Authenticated	List inbox notifications.
Notifications	POST /notifications	Admin, instructor	Send notification.
Notifications	PATCH /notifications/:id/read	Authenticated	Mark read.
Quizzes	GET /quizzes/questions	Admin, instructor	Question list.
Quizzes	POST /quizzes/generate	Admin, instructor	Generate questions.
Quizzes	GET /quizzes/sessions	Admin, instructor	List live quiz sessions.
Quizzes	POST /quizzes/start	Admin, instructor	Start quiz session.
Quizzes	GET /quizzes/student/available	Student	Available student sessions.
Quizzes	POST /quizzes/sessions/:id/accept	Student	Accept live quiz.
Quizzes	POST /quizzes/sessions/:id/submit	Student	Submit answers.
Quizzes	GET /quizzes/sessions/:id/results	Admin, instructor, student	Quiz results by role

Quizzes	PATCH /quizzes/sessions/:id/close	Admin, instructor	Close session.
Reports	GET /reports/dashboard	Admin	Dashboard totals.
Reports	GET /reports/success-center	Admin, instructor	At-risk student list.
Reports	POST /reports/success-center/:id/interventions	Admin, instructor	Add intervention note.
Reports	GET /reports/instructor-dashboard	Instructor	Instructor dashboard.
Reports	GET /reports/attendance	Admin, instructor	Attendance report.
Reports	GET /reports/performance	Admin, instructor	Performance report.
Reports	GET /reports/class-analytics/:id	Admin, instructor	Class analytics.
Search	GET /search?q=term	Admin, instructor	Global search.
CMS	GET, POST /cms/lessons	GET authenticated, POST admin	Lesson list/create.
CMS	PUT /cms/lessons/:id	Admin	Update lesson.
CMS	POST /cms/assets	Admin, instructor	Upload CMS asset.
CMS	GET, POST /cms/announcements	GET authenticated, POST admin/instructor	Announcement list/create.
CMS	GET /cms/pages	Admin	List CMS pages.
CMS	PUT /cms/pages/:slug	Admin	Update page by slug.
CMS	GET, POST /cms/quiz-questions	GET authenticated, POST admin/instructor	Question bank.
Copilot	GET /copilot/context	Authenticated	Context for selected mode.
Copilot	GET /copilot/threads	Authenticated	User threads.
Copilot	GET /copilot/threads/:id/messages	Authenticated	Thread messages.
Copilot	POST /copilot/chat	Authenticated	Create assistant response.
Copilot	GET, POST /copilot/drafts	Authenticated	Draft list/create.
Copilot	PATCH /copilot/drafts/:id/status	Authenticated with role rules	Approve/reject draft.
Copilot	GET /copilot/documents	Authenticated	List visible Copilot documents.
Copilot	POST /copilot/documents	Admin, instructor	Upload Copilot document.
Copilot	GET /copilot/audit	Admin	Audit/draft overview.
Recommendations	GET /recommendations	Authenticated	Student recommendations.
Import/Export	GET /exports/:entity? format=csv json xlsx	Admin	Export students/instructors/courses/attendance/performance.
Import/Export	POST /imports/students	Admin	Import student file.

09 UI/UX Documentation

User Flows

Flow	Steps	Outcome
Visitor Admission	Public home -> Browse courses -> Application form -> Submit -> Admin review.	Application record available in Admissions.
Admin Operations	Login -> Dashboard -> Students/Instructors/Courses -> Reports/CMS/Applications.	Institutional data is managed from one portal.
Instructor Teaching	Login -> Teaching dashboard -> Class Studio -> Assignments/Attendance/Quizzes -> Success Center.	Class delivery and student support are handled by instructor.
Student Learning	Login -> Home -> Lessons/Work/Quizzes/Documents/Grade book/Inbox.	Student sees coursework, submissions, notifications, and results.
Document Review	Student uploads -> Admin views pending -> Approve/reject -> Student sees status.	Document lifecycle is tracked.
Live Quiz	Instructor starts -> Student notified -> Student accepts/submits -> Instructor reviews results.	Realtime assessment session is recorded.

Wireframes

Public Home

Course cards and application form.

Login

Email, password, submit.

Admin Dashboard

Stats, charts, activity.

Students

Search, form, table, classes.

Class Studio

Class day editor and grading.

Student Home

Courses, work, notifications.

Design System

Element	Current Design	Guideline
Colors	Primary teal <code>hsl(184 70% 32%)</code> , accent amber <code>hsl(38 92% 56%)</code> , success green, danger red, muted slate background.	Keep role workflows consistent; avoid adding unrelated color themes.
Typography	System UI stack with Inter-style fallback.	Use concise labels, compact dashboard headings, and readable table text.

Spacing	Tailwind spacing with cards, section headers, tables, and responsive layout.	Keep dense admin pages scannable; avoid oversized marketing sections inside portal.
Buttons	Shared Button component with variants; Lucide icons in navigation/actions.	Use icons for common actions and text labels for destructive or business-critical commands.
Forms	React Hook Form/Zod patterns in pages.	Show validation errors near fields; keep required fields clear.
Tables	Shared table components for lists.	Support pagination/search for large datasets.
Cards	Used for stats, grouped forms, and content panels.	Keep cards focused on repeated records or tool panels.
Icons	Lucide icons in AppShell and controls.	Use accessible titles/tooltips where icon meaning is not obvious.

10 Security Documentation

Authentication

EduCore uses JWT access tokens signed with `JWT_ACCESS_SECRET` and refresh tokens generated with cryptographic random bytes. Refresh tokens are stored as SHA-256 hashes in MySQL and rotated on refresh. Passwords are hashed with bcrypt cost 12.

Authorization

Role	Primary Permissions
Admin	Full operational management, users, admissions, courses/classes, documents, CMS, reports, imports/exports, certificate template and revocation, Copilot audit.
Instructor	Teaching dashboard, assigned students/classes, Class Studio, assignments, grading, attendance, quizzes, notifications, success center, reports by role.
Student	Student home, lessons, work submissions, quizzes, documents, gradebook/certificates, inbox, recommendations, Copilot.
Public	Public course list and course application submission only.

Security Controls Present

- **Helmet:** default HTTP security headers.
- **CORS:** restricted to configured `CLIENT_URL`.
- **Rate limiting:** 240 requests per minute per limiter context in `app.ts`.
- **Input validation:** Zod validation middleware for many write endpoints.
- **Password hashing:** bcrypt hashes, never plain text storage.
- **Refresh token hashing:** database stores token hashes, not raw refresh tokens.
- **Upload file size limit:** Multer capped at 10 MB and sanitizes filenames.
- **Database constraints:** unique email, student code, enrollment, submission, and token hash constraints.
- **Error handling:** structured validation and application errors, duplicate handling, generic 500 message.

Security Review Findings

Risk	Severity	Observation	Mitigation
Exposed secret in repository note	High	A probable external API key appears in a plain text project note. The key is not reproduced here.	Immediately rotate/revoke the key, remove it from repository history, add secret scanning, and store secrets only in environment variables.
Development JWT defaults	High	Environment schema includes development fallback JWT secrets.	Fail startup in production if strong secrets are missing; never use defaults in production.
Token storage in localStorage	Medium	Frontend stores access and refresh tokens in localStorage.	Move refresh token to HttpOnly Secure SameSite cookie or harden XSS protections and token lifetime.
EduCore LMS & DMS Enterprise Documentation		Reported on 2024-08-20	

Upload validation	Medium	File uploads limit size and sanitize names, but no MIME allowlist/virus scanning is documented.	Add MIME/content validation, antivirus scanning, object storage, and signed file access.
Audit coverage	Medium	Activity logs exist, but not every write path is guaranteed to create audit entries.	Centralize audit logging for sensitive actions: login, role changes, grade changes, document review, certificate issue/revoke.
Swagger schema completeness	Low	OpenAPI currently lists many endpoints but has limited request/response schemas.	Expand OpenAPI components and examples for all endpoints.
Transport security	High in production	Local URLs use HTTP for development.	Use HTTPS/TLS in staging and production; secure cookies and HSTS.

Security Requirements For Production

- Enable dependency audit and secret scanning in CI.
- Configure strong secrets, SMTP credentials, and AI provider keys only in secure environment storage.
- Use HTTPS and secure cookie/session strategy.
- Limit admin access by least privilege and enforce strong password policy or SSO/MFA.
- Encrypt backups and restrict file/object storage access.
- Run penetration testing for auth, RBAC, file upload, IDOR, XSS, CSRF, and injection risk.

11 Testing Documentation

Current verification: `npm run typecheck` completed successfully for both backend and frontend during this review.

Current gap: No automated unit, integration, API, E2E, performance, or security test suite was found in the repository. Testing below is the recommended plan and initial test case catalog.

Test Plan

Type	Scope	Recommended Tools
Unit	Auth service, token hashing, validation helpers, pagination, report calculations, recommendation logic.	Vitest/Jest.
Integration	Route handlers with MySQL/Mongo test databases, RBAC, uploads, import/export.	Supertest, Testcontainers or Docker Compose test DBs.
API	OpenAPI examples and endpoint contract checks.	Postman/Newman or Schemathesis.
E2E	Public application, login, admin CRUD, instructor class studio, student submission, quiz flow.	Playwright.
Performance	Dashboard, list endpoints, search, reports, uploads.	k6, Artillery, Lighthouse.
Security	Auth, RBAC, IDOR, upload, dependency audit, secret scanning.	npm audit, gitleaks, OWASP ZAP.

Test Cases

ID	Scenario	Expected Result
TC-001	Login with valid admin credentials.	Returns user, access token, refresh token; admin portal opens.
TC-002	Login with invalid password.	Returns 401 and no tokens.
TC-003	Refresh token rotation.	Old token revoked; new token pair issued.
TC-004	Student attempts admin-only user list.	Returns 403.
TC-005	Create student with duplicate email.	Returns 409 duplicate record.
TC-006	Create course and class.	Course/class stored and visible in classes list.
TC-007	Enroll same student twice in same class.	Second enrollment rejected by unique constraint.
TC-008	Create class day and publish.	Class day appears to enrolled student content flow.
TC-009	Upload submission file under 10 MB.	Submission is stored with file URL.
TC-010	Upload submission file over 10 MB.	Upload fails.

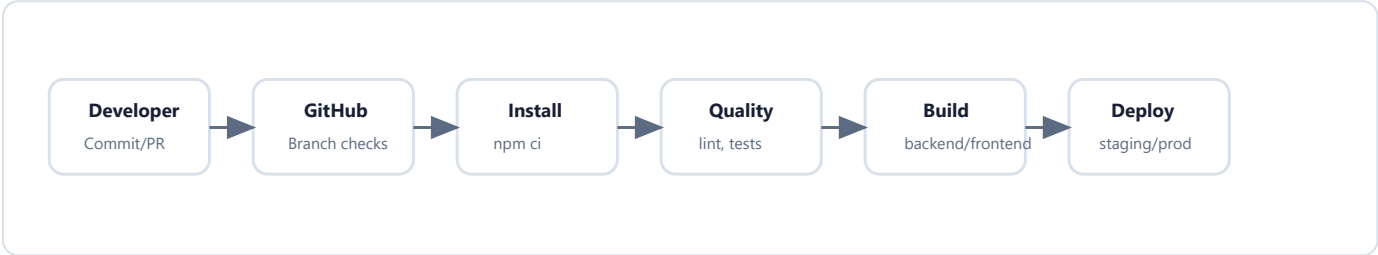
TC-011	Grade submission.	Grade and feedback persist; gradebook updates.
TC-012	Bulk attendance save.	One attendance record per selected student/day is created or updated.
TC-013	Student document review.	Admin status change updates student-visible document state.
TC-014	Send notification to role.	Notification persists and matching socket room receives event.
TC-015	Start quiz and student submits.	Attempt score is calculated and results are visible by role.
TC-016	Issue certificate for eligible student.	Certificate has unique verification code.
TC-017	Global search with valid query.	Grouped results return across available entities.
TC-018	Export students as CSV.	CSV response downloads with expected columns.
TC-019	Import invalid student sheet.	Invalid rows are rejected or reported.
TC-020	Production startup without secrets.	Application fails safe rather than using dev defaults.

12 DevOps Documentation

Current DevOps Assets

- `docker-compose.yml` defines MySQL, MongoDB, backend, and frontend services.
- Backend and frontend each include Dockerfiles.
- Root npm workspace scripts support `dev`, `build`, `typecheck`, and `seed`.
- Backend exposes `GET /health` for basic service health.

Recommended CI/CD Pipeline



Environments

Environment	Purpose	Data	Controls
Development	Local implementation and demo work.	Seed/demo data.	Local secrets, hot reload.
Testing	Automated unit/integration/E2E tests.	Disposable test data.	Reset DB per test run.
Staging	Production-like acceptance testing.	Masked or synthetic data.	HTTPS, monitoring, backups.
Production	Live use.	Real academic/user data.	Least privilege, backups, audit, alerting, incident response.

13 Deployment Guide

Server Requirements

Component	MVP Minimum	Production Recommendation
CPU	2 cores	4+ cores or horizontally scalable containers.
RAM	4 GB for full stack locally.	8+ GB split across app/database services.
Storage	20 GB for code, dependencies, local DB and uploads.	Managed database storage plus object storage and backups.
OS	Windows with XAMPP or Linux with Docker.	Linux container host or managed app platform.
Runtime	Node.js compatible with project dependencies.	Pinned Node LTS version in CI and containers.
Databases	MySQL 8 and MongoDB 7.	Managed MySQL/MongoDB with automated backups.

Local Installation Steps

```
1. Copy .env.example to backend/.env and set real secrets.
2. Create MySQL schema:
  mysql -h 127.0.0.1 -P 3306 -u root < backend/database/schema.sql
3. Install dependencies:
  npm install
4. Seed demo data:
  npm run seed
5. Run full stack:
  npm run dev
6. Open frontend:
  http://localhost:5173
7. Open API docs:
  http://localhost:4000/api/docs
```

Docker Deployment Steps

```
1. Create backend/.env from .env.example.
2. Replace all production secrets and database credentials.
3. Run:
  docker compose up --build
4. Verify:
  http://localhost:5173
  http://localhost:4000/health
  http://localhost:4000/api/docs
```

Environment Variables

Variable	Purpose
NODE_ENV, PORT, CLIENT_URL	Runtime environment, API port, allowed frontend origin.
JWT_ACCESS_SECRET, JWT_REFRESH_SECRET	JWT signing secrets. Must be strong in production.
JWT_ACCESS_EXPIRES_IN, JWT_REFRESH_EXPIRES_IN	Token lifetime configuration.

UPLOAD_DIR	Local file upload path.
DB_HOST, DB_PORT, DB_USER, DB_PASSWORD, DB_NAME	MySQL connection.
MONGO_URI	MongoDB connection.
GROQ_API_KEY, GROQ_MODEL, GROQ_BASE_URL	Optional Groq-compatible AI provider.
OPENAI_API_KEY, OPENAI_MODEL, OPENAI_BASE_URL	Optional OpenAI-compatible AI provider.
SMTP_HOST, SMTP_PORT, SMTP_SECURE, SMTP_USER, SMTP_PASS, MAIL_FROM	Email delivery.
CLOUDINARY_CLOUD_NAME, CLOUDINARY_API_KEY, CLOUDINARY_API_SECRET	Future/optional file storage integration.

14 Maintenance Documentation

Monitoring

Area	Current State	Recommended Tooling
Application health	<code>/health</code> returns service OK.	Uptime checks and container health probes.
Application logs	Morgan request logging and console errors.	Structured JSON logs shipped to Loki/ELK/CloudWatch.
Error tracking	No dedicated tracker in repo.	Sentry or equivalent with source maps.
Metrics	No metrics endpoint in repo.	Prometheus + Grafana or platform metrics.
Analytics	Reports exist from app data.	Add product analytics for adoption and usage.

Backup Strategy

- **MySQL:** nightly logical dump, transaction-safe backup where available, 30-day retention, encrypted storage.
- **MongoDB:** nightly `mongodump` or managed snapshots, 30-day retention, encrypted storage.
- **Uploads:** object storage versioning or filesystem snapshot; include submissions, documents, CMS/Copilot assets.
- **Restore tests:** monthly restore drill into staging environment.

Disaster Recovery

Scenario	Impact	Recovery Action	Target
Backend container failure	API unavailable.	Restart service; inspect logs; redeploy last stable image.	RTO 30 minutes.
MySQL failure	Core records unavailable.	Restore latest backup or promote managed replica.	RTO 4 hours, RPO 24 hours unless upgraded.
MongoDB failure	Content, notifications, quizzes, documents metadata unavailable.	Restore snapshot/dump.	RTO 4 hours, RPO 24 hours unless upgraded.
Secret leak	Credential compromise risk.	Revoke/rotate, audit access, purge history, redeploy.	Immediate response.
File storage loss	Uploads unavailable.	Restore from object storage versioning or filesystem snapshot.	RTO 4-8 hours.

15 Project Management Documentation

Roadmap

Phase	Scope	Exit Criteria
Phase 1 - MVP	Auth, RBAC, student/instructor/course/class records, assignments, attendance, reports, documents, CMS basics, notifications, seed data, Docker, Swagger.	Demo users complete core workflows; typecheck and core tests pass.
Phase 2 - Advanced Operations	Admissions automation, class analytics, success center, live quizzes, certificate designer, stronger import/export, audit logging, better OpenAPI.	Role workflows are test-covered and ready for staging pilot.
Phase 3 - AI and Personalization	Copilot provider hardening, recommendation rules, draft approvals, knowledge documents, student support automation.	AI outputs are auditable, approved where needed, and privacy reviewed.
Phase 4 - Production Hardening	CI/CD, tests, security scan, secret rotation, backups, monitoring, object storage, HTTPS, runbooks.	Production readiness checklist signed off.

Risk Register

Risk	Impact	Probability	Mitigation
Exposed credentials	High	Medium	Rotate exposed key, add secret scanning, remove secrets from repo/history.
No automated tests	High	High	Add unit, integration, API, and E2E suites before production.
Data loss	High	Medium	Automated backups and restore drills for MySQL, MongoDB, uploads.
RBAC gaps	High	Medium	Route-level authorization tests and IDOR testing.
File upload abuse	Medium	Medium	MIME validation, AV scanning, object storage, size/type policies.
Realtime scaling limitations	Medium	Low initially	Use Socket.IO adapter/backplane for multi-instance production.
Scope creep	Medium	High	Freeze MVP, use backlog priorities, defer non-critical integrations.
Deployment drift	Medium	Medium	Use container images, pinned envs, CI/CD and IaC.

Sprint Plan

1. Setup: repository, frontend/backend TypeScript, MySQL, MongoDB, Docker, schema, seed.

2. Authentication: register, login, refresh, logout, JWT middleware, RBAC, protected frontend routes.
3. DMS: students, instructors, classes, schedules, attendance, analytics.
4. LMS: courses, lessons, assignments, submissions, grading, progress.
5. Advanced features: realtime notifications, search, import/export, CMS, recommendations.
6. Finalization: responsive UI, Swagger, seed data, testing, bug fixing, presentation/report.

16 Change Log

Version	Date	Changes	Status
0.1	Planning	Project master plan, initial scope, documentation structure.	Completed
0.3	Foundation	Monorepo, React frontend, Express backend, TypeScript, Tailwind, Docker, environment template.	Completed
0.5	Authentication	JWT auth, refresh tokens, RBAC middleware, protected frontend routes.	Completed
0.7	DMS/LMS Core	Users, students, instructors, courses, classes, enrollments, assignments, submissions, attendance, gradebook.	Completed
0.9	Advanced Features	Applications, documents, certificates, CMS, announcements, quizzes, notifications, reports, search, import/export, Copilot.	Completed/MVP
1.0	May 26, 2026	Enterprise documentation PDF generated from code review; typecheck passed; security/testing gaps documented.	Review Draft

17 User Documentation

User Guide

User	How To Use EduCore
Visitor / Applicant	Open the public homepage, review courses, fill in the application form, and wait for admissions review.
Student	Login with student account, use Home for current classes and notifications, open Lessons for class content, Work for assignments/submissions, Quizzes for live quizzes, Documents for uploads, Gradebook for results/certificates, and Inbox for messages.
Instructor	Login and use Teaching for summary, Class Studio for class-day content and grading, Assignments for assignment health, Attendance for roster status, Quizzes for live sessions, Success Center for interventions, Notifications for messages, and Search for records.
Admin	Login and use Dashboard, Students, Instructors, Applications, Courses, Documents, Gradebook, CMS, Reports, Notifications, Search, imports, and exports to manage the institution.

Admin Guide

1. Seed or create admin account.
2. Create instructors and reset credentials as needed.
3. Create students or enroll applicants after approval.
4. Create courses and classes with schedules.
5. Enroll students into classes.
6. Review uploaded documents and update statuses.
7. Configure CMS pages, lessons, announcements, quiz questions, and certificate template.
8. Monitor dashboard, reports, success center, and exports.

FAQ

Question	Answer
Where are default demo accounts?	The README lists admin, instructor, and student demo accounts created by <code>npm run seed</code> .
Where is the API documentation?	Run the backend and open <code>http://localhost:4000/api/docs</code> .
Can students create courses?	No. Course creation is admin-only in the current backend.
Can instructors grade submissions?	Yes, through assignment submission grading routes and related UI.
Where are files stored?	Currently local upload folders. Production should move to managed object storage.
Does the project have automated tests?	No automated test suite was found during this review. TypeScript typecheck passes.

18 Technical Documentation Structure

Recommended enterprise documentation layout for this project:

```
/docs
|-- 01_Project_Charter
|   |-- Project_Charter.md
|-- 02_Business_Requirements
|   |-- BRD.md
|-- 03_Software_Requirements
|   |-- SRS.md
|-- 04_User_Stories
|   |-- User_Stories.md
|-- 05_Use_Cases
|   |-- Use_Cases.md
|-- 06_Architecture
|   |-- System_Architecture.md
|   |-- Infrastructure.md
|   |-- Deployment_Architecture.md
|-- 07_Database
|   |-- ERD.md
|   |-- Tables.md
|   |-- Mongo_Collections.md
|-- 08_API
|   |-- OpenAPI.yaml
|   |-- Postman_Collection.json
|   |-- Endpoint_Catalog.md
|-- 09_UI_UX
|   |-- User_Flows.md
|   |-- Wireframes.md
|   |-- Design_System.md
|-- 10_Security
|   |-- Security_Architecture.md
|   |-- Threat_Model.md
|-- 11_Testing
|   |-- Test_Plan.md
|   |-- Test_Cases.md
|-- 12_DevOps
|   |-- CI_CD.md
|   |-- Environments.md
|-- 13_Deployment
|   |-- Deployment_Guide.md
|-- 14_Maintenance
|   |-- Monitoring.md
|   |-- Backup_And_Recovery.md
|-- 15_Project_Management
|   |-- Roadmap.md
|   |-- Risk_Register.md
|-- 16_Changelog
|   |-- CHANGELOG.md
|-- 17_User_Guides
|   |-- User_Guide.md
|   |-- Admin_Guide.md
|   |-- FAQ.md
```

Review Summary

Area	Status	Next Action
------	--------	-------------

Architecture	Strong MVP separation with React frontend, Express API, MySQL, MongoDB, Docker.	Harden deployment, add queue/cache/object storage if production traffic grows.
Features	Broad LMS/DMS feature coverage across roles.	Prioritize polish and acceptance tests for most-used workflows.
Security	Good foundations; important gaps remain.	Rotate exposed secret, fix production secret policy, improve token/upload hardening.
Testing	Typecheck passes; automated tests absent.	Add unit/integration/E2E suites and CI gates.
Documentation	Existing docs are partial; this PDF consolidates enterprise sections.	Split this document into the recommended docs tree if long-term maintenance is needed.

Reviewed Local Sources

[README.md](#), [.env.example](#), [docker-compose.yml](#), [docs/project-plan.md](#), [docs/erd.md](#), [docs/api-docs.md](#), [docs/database-setup.md](#), [project-management/sprint-plan.md](#), [backend/package.json](#), [backend/src/app.ts](#), [backend/src/server.ts](#), [backend/src/config/env.ts](#), [backend/src/modules/*](#), [backend/src/database/*](#), [backend/database/*.sql](#), [backend/docs/openapi.yaml](#), [frontend/package.json](#), [frontend/src/App.tsx](#), [frontend/src/components/layout/AppShell.tsx](#), [frontend/src/pages/*](#), [frontend/src/services/api.ts](#), and [frontend/src/types/index.ts](#).